Artificial Models for Multimodal Creativity
Winter semester 2026/27 · Sound, symbols, sensors

Bodies as Data

LESSON 2 · CAMERAS, CONVOLUTIONAL NETWORKS, POSES AND GESTURES

Prof. Dr. Alessandro Anatrini

Hochschule für Musik und Theater Hamburg
Conservatorio di Musica “G. Nicolini”, Piacenza
Accademia di Belle Arti di Brera, Milano

Today

1	Cameras in music before deep learning; convolutional networks; what MediaPipe and YOLO compute from an image	55 min
2	From landmarks to features; classification with nearest neighbours; maps of 63 dimensions in two; gestures in time	60 min
3	Exercise 2: a webcam, a hand, three ways to play with it: poses that trigger sounds, a map of hand shapes, a gesture that is followed while it happens	85 min
4	Discussion: whose bodies are in the training data, and what the law says about cameras and bodies	20 min

From three numbers to sixty-three

EXERCISE 1

The phone sent three numbers: x and y of a touch, and its radius. The network had one hidden layer of 6 units and three outputs (carrier, ratio, index):
(3 × 6 + 6) + (6 × 3 + 3) = **45 parameters**.

Every input was a number you could read and change by hand.

EXERCISE 2

A webcam image of 640 × 480 pixels in colour is 921 600 numbers; at 30 frames per second, 27.6 million numbers per second.

A hand tracker reduces each frame to 21 points with three coordinates each: **63 numbers**. That reduction is done by convolutional networks trained on tens of thousands of photographs of hands.

Two models are now chained: one that someone else trained, which turns pixels into points, and one that you train, which turns points into sound.

Seeing machines, 1959–2026

	MODELS OF VISION 1959–1988	CONVOLUTION, FEATURES 1989–2011	DEEP LEARNING 2012–2019	ON THE DEVICE 2020–2026
Computer vision	**1959–62** Hubel and Wiesel: cells in the visual cortex respond to edges in a small field **1980** Fukushima's neocognitron: layers of local feature detectors	**1989** LeCun: a convolutional network trained with backpropagation reads zip codes **1998** LeNet-5 reads bank cheques **2001** Viola and Jones: real-time face detection with hand-designed features **2011** Kinect: body skeleton from a depth camera	**2012** AlexNet **2015** ResNet: 152 layers **2016** YOLO: detection in one pass **2017** OpenPose: skeletons of several people in real time **2019** MediaPipe released as open source	**2020** MediaPipe Hands, BlazePose on phones; Vision Transformer **2026** YOLO26
Music and art	**1975** Krueger, *Videoplace*: a projected silhouette interacts with graphics **1986** Rokeby, *Very Nervous System*, Venice Biennale: movement in front of a camera plays music	**2000** EyesWeb (Camurri, Genoa): movement analysis for dance and music **2010–11** the Kinect enters performance and installation	**2018** PoseNet runs in the web browser; ml5.js makes it available to artists	**2023** Blankensmith's MediaPipe plugin for TouchDesigner, free

Features designed by hand

In David Rokeby's *Very Nervous System* (1982–91) a video camera watched the space; the program compared successive frames and measured how much changed in each region of the image. Amount and place of the movement controlled synthesisers.

In Max: `jit.op @op absdiff` between the current and the previous frame, then `jit.3m` for the mean change, computed for each region of the image.

Patch: `max/examples/12-frame-difference.maxpat`: four strips of the image, four tones.

The Kinect (2010) added a depth camera. Its body tracker classifies every depth pixel into one of 31 body parts with a random forest, using differences of depth between nearby pixels, and was trained on synthetic depth images of bodies.

Both are classical machine learning in the sense of lesson 1: people chose what to measure (change between frames, depth differences); a program, or a learning algorithm, worked on those measures.

Rokeby, *Very Nervous System*, documentation at davidrokeby.com; Shotton et al., CVPR 2011.

Pixels into a multilayer perceptron

In lesson 1, a layer of 1000 units connected to every pixel of a 224 × 224 colour image needed about 150 million weights.

The number is not the only problem. When the hand moves ten pixels to the right, every input of the network changes, and the network must have learned the hand separately at each position.

Two properties of images suggest a different structure. Neighbouring pixels belong together, so a unit can look at a small region. And an edge or a fingertip looks the same wherever it appears, so the same weights can be used at every position.

Hubel and Wiesel (1959, 1962) found cells in the visual cortex of the cat that respond to edges of one orientation in a small part of the visual field. Fukushima's neocognitron (1980) built a network from such local detectors. LeCun (1989) trained one with backpropagation to read handwritten zip codes.

FULLY CONNECTED



3 units × 7 pixels: 21 weights, all different

CONVOLUTIONAL



each unit sees 3 neighbouring pixels; the same 3 weights (3 colours) everywhere

One filter, computed by hand



The filter is placed on a 3 × 3 region of the image; each pixel is multiplied by the weight above it and the nine products are added. In the first position the left column gives 0, the right column 1 + 1 + 1: the output is 3.

Moved one pixel at a time, the filter gives 3 where the dark and the bright region meet and 0 where the image is uniform: it detects vertical edges from dark to bright. A 5 × 5 image gives a 3 × 3 feature map.

The nine weights are not chosen by hand. They are parameters, learned by backpropagation like the weights of Exercise 1; the network finds which patterns are worth detecting.

The same filter on a photograph

PHOTOGRAPH, GREYSCALE



VERTICAL EDGES



HORIZONTAL EDGES



The filter of the previous slide, moved over every position of a photograph of 360 × 270 pixels. Dark means a large output, positive or negative. The first filter responds to vertical edges, such as the ends of the fingers; turned by 90°, it responds to horizontal edges, such as the outline of the fingers and of the arm.

Two filters give two feature maps: two channels. A layer of a convolutional network computes dozens of them at once, with weights learned from the data.

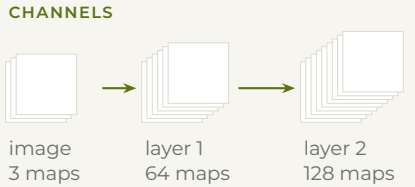
Photograph: MediaPipe sample image `woman_hands.jpg`, in greyscale; filters computed with numpy.

Many filters, many layers

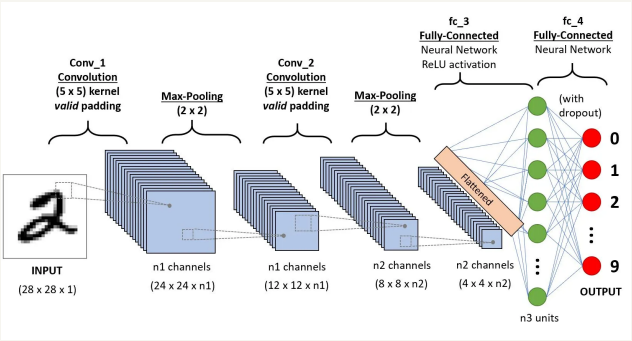
A convolutional layer applies many filters to the same input; each produces one feature map, called a *channel*. A colour image has 3 channels; after a layer of 64 filters there are 64.

A filter in the next layer covers all channels of its region at once. A layer of 128 filters of 3 × 3 over 64 channels has 128 × (3 × 3 × 64 + 1) = 73 856 parameters, independent of the size of the image.

Because each layer looks at the feature maps of the previous one, the region of the original image that influences a unit grows from layer to layer: a few pixels in the first layer, a finger in a middle layer, the whole hand in the last ones.



Pooling and classification



A network that classifies handwritten digits. A 5 × 5 filter without padding reduces 28 × 28 pixels to 24 × 24. **Max pooling** keeps the largest value of each 2 × 2 block: half the width and height, and a small shift of the input changes the result less. After two convolutions and two poolings, 4 × 4 values per channel remain; they are **flattened** into one list and passed to fully connected layers, an MLP, whose ten outputs give one probability per digit.

S. Saha, “A comprehensive guide to convolutional neural networks”, 2018.

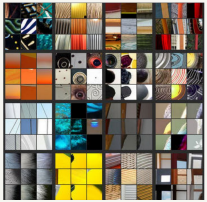
What the layers learn

LAYER 1



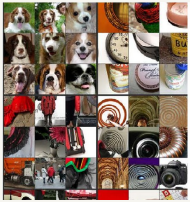
edges, patches of colour

LAYER 2



textures, stripes, circles

LAYER 4



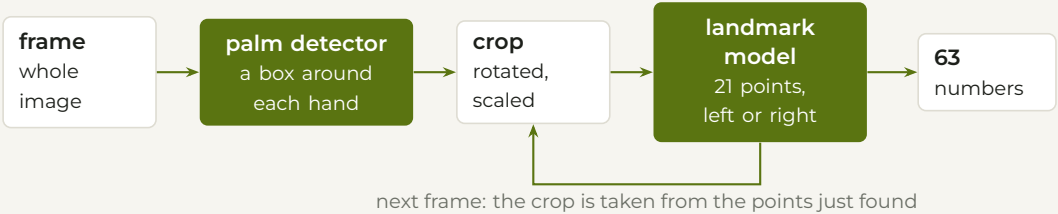
parts of objects: faces of dogs, wheels

Zeiler and Fergus (2014) displayed, for units of a network trained on ImageNet, the image regions that activate them most: each block of 3 × 3 photographs belongs to one unit.

None of these detectors was designed. They result from the training data and the task: a network trained on photographs of hands develops detectors for fingers and knuckles. The first layers of a network trained on a large dataset are useful for other images too. New detectors are often built on them and only the last layers are trained again, which needs far fewer examples (*transfer learning*).

Zeiler & Fergus, “Visualizing and understanding convolutional networks”, ECCV 2014, figure 2 (details).

MediaPipe: two networks in sequence



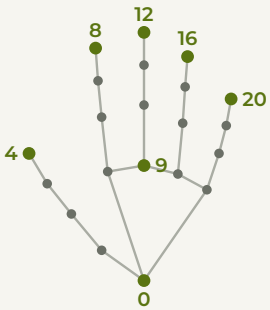
MediaPipe is a framework by Google, open source since 2019. Its Hand Landmarker runs two convolutional networks. A palm detector searches the whole image: palms are nearly rigid and easier to find than hands with moving fingers. A second network places 21 points in the cropped region.

While the hand stays in view, the crop for the next frame is computed from the current points and the detector is skipped; it runs again when the hand is lost. This keeps tracking fast enough for a laptop processor or a phone.

The landmark model was trained on about 30 000 real photographs, annotated by hand with 21 points each, and on rendered images of a synthetic hand model over varied backgrounds.

MediaPipe Hand Landmarker documentation; Zhang et al., “MediaPipe Hands”, arXiv:2006.10214, 2020.

Twenty-one landmarks



Point 0 is the wrist; 4, 8, 12, 16 and 20 are the fingertips from thumb to little finger; 9 is the base of the middle finger. Each finger has four points; the numbering is the same in every frame and for every hand.

x and **y** are divided by the width and the height of the image: 0 to 1 from left to right and from top to bottom. **z** is the depth relative to the wrist; smaller values are closer to the camera.

A second set, the *world landmarks*, gives the same points in metres, with the origin at the geometric centre of the hand. The tracker also reports whether the hand is a left or a right one, and a confidence for detection, presence and tracking (each threshold 0.5 by default).

MediaPipe Hand Landmarker documentation, Python guide, section on results.

Hands, bodies, faces

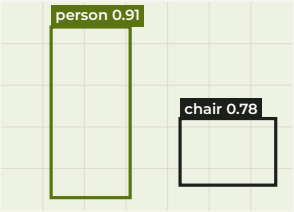
MediaPipe task	Points	Per point	Origin of depth
Hand Landmarker	21	x, y, z	wrist
Pose Landmarker	33	x, y, z, visibility	midpoint of the hips
Face Landmarker	478	x, y, z	—

The Pose Landmarker (BlazePose, 2020) covers face, shoulders, arms, hands, hips, legs and feet. *Visibility* is the estimated probability that a point is visible and not hidden by the body or out of the frame. It comes in three sizes, Lite, Full and Heavy: larger models are more accurate and slower.

A body gives 33 × 3 = 99 numbers per frame, a face 1434. Most of them change together: when the hand closes, all fingertips move. Part of today is about finding the few directions in which such data actually vary.

MediaPipe Pose and Face Landmarker documentation; Bazarevsky et al., “BlazePose”, arXiv:2006.10204, 2020.

YOLO: objects and people in one pass



YOLO, “you only look once” (Redmon et al., 2016), divides the image into a grid. For each cell a single convolutional network predicts boxes, a confidence that a box contains an object, and the object’s class, in one pass over the image. Earlier detectors classified thousands of candidate regions one by one.

The current versions are developed by Ultralytics; YOLO26 was released in January 2026. The standard models recognise the 80 classes of the COCO dataset, among them *person*. The same family offers segmentation and pose estimation with the 17 body keypoints of COCO.

The Ultralytics code and models are licensed under AGPL-3.0: a work that includes them and is distributed, or offered over a network, must publish its own source code under the same licence, unless a commercial licence is bought.

Redmon et al., CVPR 2016; docs.ultralytics.com, YOLO26.

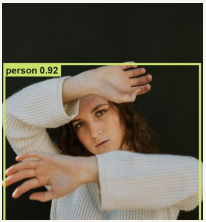
The same photograph, three models

MEDIAPIPE HAND LANDMARKER



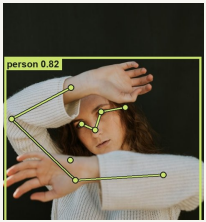
21 points for each hand, with every finger joint
2 hands × 63 numbers

YOLO26 DETECTION



a box, a class and a confidence: 0.92 means 92 % sure it is a person
6 numbers per object

YOLO26 POSE



a box and 17 body points: eyes, ears, shoulders, elbows, wrists, hips, knees, ankles
51 numbers per person

MediaPipe sees the shape of one hand: the fingers, and so the hand poses of Exercise 2. YOLO sees people and objects in the room: how many, where, and the posture of the body; its skeleton ends at the wrist.

Photograph: MediaPipe sample image. Computed with MediaPipe 1.1.1 and Ultralytics 8.4 (`yolo26n`, `yolo26n-pose`); pose points with confidence above 0.5.

Labelled images

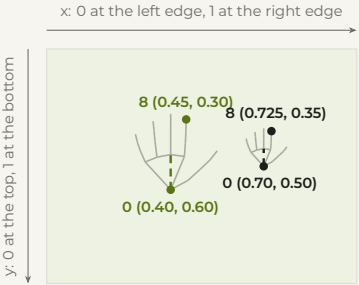
Both trackers are supervised models. Their targets are points and boxes drawn by people on photographs.

COCO (Lin et al., 2014) contains 328 000 images taken from Flickr with 2.5 million labelled object instances. The outlines were drawn by crowd workers paid per task through Amazon Mechanical Turk. Seventeen body keypoints per person were added later for a pose benchmark.

A tracker knows bodies as they appear in these photographs: their lighting, their skin tones, their clothes, the poses people take in front of a camera. For a hand or a body unlike the training images, the points become less accurate or the detection fails. This is a property of the data, and it is inherited by everything you build on top of the tracker.

Lin et al., “Microsoft COCO: common objects in context”, ECCV 2014.

The same hand at two distances



	Near the camera	Further away
wrist (point 0)	(0.40, 0.60)	(0.70, 0.50)
index tip (point 8)	(0.45, 0.30)	(0.725, 0.35)
tip minus wrist	(0.05, −0.30)	(0.025, −0.15)
palm length, 0 to 9 (dashed)	0.15	0.075
divided by palm length	**(0.33, −2.0)**	**(0.33, −2.0)**

Points of the MediaPipe Hand Landmarker. Each pair is (x, y): a position in the image, as a fraction of its width and height. They are not confidences.

The raw coordinates say where the hand is in the image and how large it appears. For recognising a hand shape, both are noise: the same pose should give the same numbers anywhere in the frame.

Subtracting the wrist removes the position; dividing by the length of the palm removes the distance from the camera. What remains describes the shape of the hand. For an instrument that should react to where the hand is, keep the wrist position as additional features.

Two details of the coordinates

WIDTH AND HEIGHT

x is divided by the width of the image, y by its height. In a 16:9 image a step of 0.1 in x is 1.78 times longer than a step of 0.1 in y. A hand rotated by 90 degrees then changes its proportions. Multiplying x (and z, which uses the same scale) by width/height before computing distances corrects this.

JITTER

From frame to frame the points move by a few pixels even when the hand is still. Averaging the last few frames smooths them but adds delay. The 1 € filter (Casiez et al., 2012) smooths strongly when a point moves slowly and little when it moves fast, so that slow movements are steady and fast ones are not delayed.

At 30 frames per second a new set of points arrives every 33 ms. The camera, the tracker and the network together add tens of milliseconds: enough to play continuous changes of sound, less suited to precise rhythmic attacks.

Features computed from the points

From the 21 points of the MediaPipe Hand Landmarker, the tracker of Exercise 2; numbers in brackets are point numbers (0 is the wrist). YOLO pose has no finger points: of these features, only position and velocity can be computed from it, with its wrist points.

Feature	Computed as	Describes
hand shape	the 21 points relative to the wrist, divided by the palm length (63 numbers)	the pose, independent of position and size
pinch	distance between thumb tip (4) and index tip (8)	a continuous control, 0 when the fingers touch
openness	mean distance of the five fingertips from the wrist	fist to open hand
position	wrist x and y	where the hand is in the frame
velocity	difference between two successive frames	how fast the hand moves

A few designed features (pinch, openness) are easy to map by hand. The 63 numbers of the hand shape carry more information, but only a model can use them.

Two kinds of scaling

GEOMETRIC

Relative to the body: subtract a reference point, divide by a length of the body. It uses the points of one frame only, and makes poses comparable across positions, distances and people with different hand sizes.

STATISTICAL

Relative to the dataset: `fluid.normalize~` or `fluid.standardize~`, as in Exercise 1. It uses all recorded examples, and gives every feature a comparable range before distances or weighted sums are computed.

The order is geometric first, statistical second. The statistics of raw coordinates would mostly describe where you stood while recording.

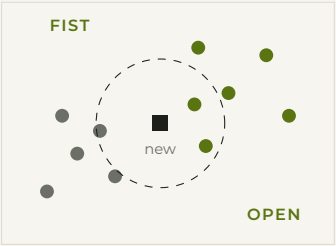
Labels instead of numbers

In Exercise 1 the target of each example was a list of numbers. To trigger a sound file, start a section of a piece or change a preset, the target is a category: *fist*, *open*, *point*, *pinch*.

In FluCoMa the categories are stored in a `fluid.labelset~`: one label per identifier, with the same identifiers as the dataset of features. A classifier is trained with `fit` *dataset labelset* and answers `predictpoint` with a label.

Identifier	**`fluid.dataset~`: 63 features**	**`fluid.labelset~`**
`ex1`	0.00 0.00 0.00 −0.48 −0.31 …	fist
`ex2`	0.00 0.00 0.00 −0.52 −0.29 …	fist
`ex3`	0.00 0.00 0.00 −0.71 −0.45 …	open

k nearest neighbours



Two of 63 features, for illustration.

Every example is a point in a space with one axis per feature. A new hand shape is classified by looking for the *k* examples closest to it and taking the most frequent label among them.

Here, with *k* = 3, two of the three nearest examples are *open*: the answer is *open*. With `@weight 1` (the default) nearer neighbours count more than further ones.

Training only stores the examples in a search tree; it takes no time and has no learning rate. In `fluid.knnclassifier~` *k* is `@numneighbours`, 3 by default. The method goes back to Fix and Hodges (1951).

A classifier always answers

A classifier with the labels *fist*, *open* and *point* gives one of the three for every input, also for a hand that is scratching a nose or moving between two poses.

Three remedies, which can be combined:

1. Record a class *rest*, with examples of everything that should not trigger anything: the relaxed hand, transitions, the hand half out of the frame.
2. Measure the distance to the nearest example with `fluid.kdtree~` (`knearestdist`) and ignore the answer when it is too large.
3. Accept a change of label only after it has been the same for several frames, for example five (about 170 ms at 30 frames per second).

Nearest neighbours or a network

	`fluid.knnclassifier~`	`fluid.mlpclassifier~`
Training	stores the examples, instantly	adjusts weights over many epochs
Prediction	searches the stored examples; slower with many examples	a fixed number of operations
Few examples	works from one example per class	needs more examples, and settings
Between classes	the border lies halfway between examples	a smooth border learned from all examples
Settings	`numneighbours`, `weight`	as `fluid.mlpregressor~`, without output activation

For a dozen poses recorded during rehearsal, nearest neighbours are usually sufficient and are easier to correct: a wrong answer is fixed by adding or deleting an example.

Sixty-three dimensions

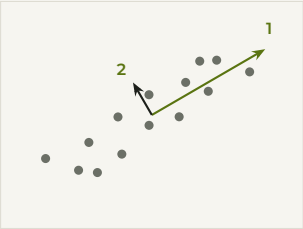
A space with one axis per feature cannot be drawn beyond three axes. To see how the recorded hand shapes relate to each other, the 63 numbers have to be reduced to two or three.

For hands this loses less than it seems. Santello, Flanders and Soechting (1998) asked five people to shape the hand as if to grasp and use a large number of familiar objects, and measured 15 joint angles. Two combinations of angles accounted for more than 80% of the variation: the joints of a hand move together, not independently.

Reducing dimensions is unsupervised learning (lesson 1, four ways of learning): there are no targets; the method finds a structure in the examples themselves.

Santello, Flanders & Soechting, “Postural hand synergies for tool use”, *Journal of Neuroscience* 18(23), 1998.

Principal component analysis



PCA (Pearson 1901, Hotelling 1933) looks for the direction along which the examples vary most: the first principal component. The second is the direction of largest remaining variation, at right angles to the first, and so on.

Keeping the first two components is a rotation of the space followed by a projection onto a plane. It is linear: straight lines stay straight, and a point can be mapped back to all 63 features (`inversetransformpoint`).

In `fluid.pca~`, `@numdimensions` sets how many components are kept (default 2). Like `fluid.normalize~`, it is fitted on a dataset and then applied to single points with `transformpoint`.

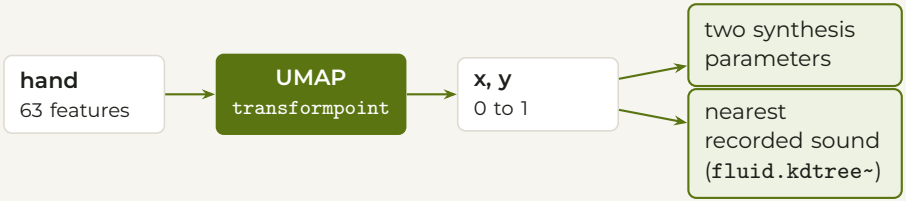
UMAP

UMAP (McInnes, Healy and Melville, 2018) connects every example to its nearest neighbours in the original space, then arranges the examples in two dimensions so that the same examples are neighbours there. It keeps neighbourhoods, not distances; the result can be curved and folded in ways no projection allows.

`fluid.umap~`	Default	Effect
`numneighbours`	15	few: many small clusters; many: the overall shape
`mindist`	0.1	how closely points may be packed in the map
`numdimensions`	2	dimensions of the map
`iterations`	200	steps of the arrangement

Two caveats. The distance between two clusters in the map means little; only membership of a neighbourhood does. And each fit starts from a random arrangement, so the same data give a differently oriented map each time. New points are placed into an existing map with `transformpoint`, which t-SNE (2008), the earlier standard method, could not do.

A map as an instrument



Fitted on a few hundred hand shapes, the map places similar shapes near each other. Moving the hand moves a point across the map, shown with `fluid.plotter`. Its two coordinates can be used like the touch x and y of Exercise 1, but the axes have no names: they are whatever separates your shapes best.

FluCoMa's examples use the same technique for sound: descriptors of hundreds of short sounds are reduced to two dimensions and the map is played by moving through it. In block 4 the map will not only arrange existing sounds; a model will generate sound from any point of it (a latent space).

Poses and gestures

A pose is one frame: 63 numbers. A gesture is a sequence of frames: a circle drawn in the air in two seconds at 30 frames per second is a tensor of shape (60, 63), or (60, 2) if only the index fingertip is kept.

Three problems do not exist for poses. The same gesture is performed faster or slower, so sequences of different lengths must be compared. It is not known in advance when a gesture begins and ends. And for music it is often too late to recognise a gesture once it is finished: the sound should react while it happens.

Sequences return in block 3, where the model generates them; today it only recognises and follows them.

Dynamic time warping



Dynamic time warping (Sakoe and Chiba, 1978, for spoken words) compares a new sequence with a recorded template by stretching and compressing time. It finds the pairing of frames, in order, with the smallest total distance; that distance says how similar the two gestures are regardless of speed.

Wekinator uses it to recognise gestures from a few recorded examples: it compares the most recent frames with each template and reports the closest. It does not say how far through a gesture the performer is.

Following a gesture



A hidden Markov model represents a recorded gesture as a chain of states, each standing for a part of the gesture. At every frame the model can stay in its state or move on to the next; each state has learned which features to expect. From the incoming frames it estimates the probability of being in each state.

This gives two answers at every frame, not only at the end: how likely each recorded gesture is (*likelihood*), and how far through it the performer is (*progress*, 0 to 1). Bevilacqua's gesture follower at IRCAM (2010) was built for this use; MuBu's `mubu.hhmm` (hierarchical HMM, Françoise et al., 2014) connects several gestures in one model.

Progress can drive the playback position of a sound: a slower gesture stretches the sound, a reversed one stops it.

Bevilacqua et al., LNCS 5934, 2010; Françoise, Schnell, Borghesi & Bevilacqua, NIME 2014; Rabiner 1989.

MuBu

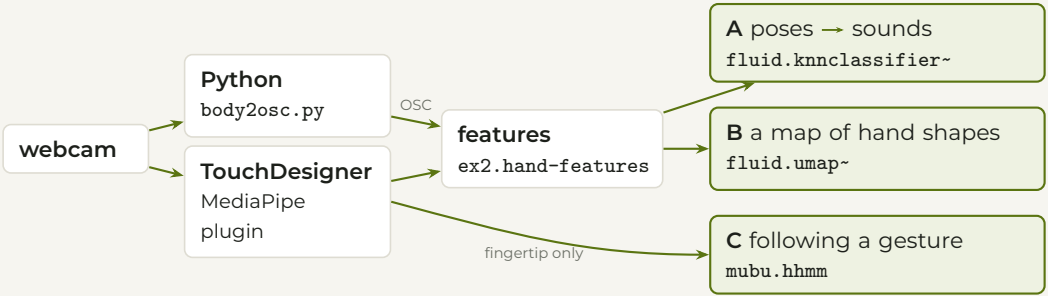
MuBu (“multi-buffer”) is a package for Max developed by the Sound Music Movement Interaction team at IRCAM, free from the Max Package Manager (version 1.11). A `mubu` container holds several tracks of time-stamped data, for example audio, descriptors and motion, in synchrony.

`mubu.record`	records incoming lists into a track, one buffer per example
`mubu.gmm`	recognises static poses (Gaussian mixture models)
`mubu.hhmm`	recognises and follows gestures in time: outputs `likelihood` per gesture and `progress`
`mubu.xmm`	as `mubu.hhmm`, and also outputs continuous parameters learned together with the movement

Each buffer of the container is one recorded example; by default its label is the buffer number. `train` fits one model per label; `@play 1` starts recognition. MuBu exists only for Max.

Schnell et al., ICMC 2009; MuBu for Max 1.11 reference.

Exercise 2: a hand in front of the camera



The tracker runs outside Max and sends the 21 points of one hand over OSC, to port 9001, address `/hand/raw`, 63 numbers per frame. In Max the abstraction `ex2.hand-features` turns them into the hand-shape features; part C uses only the index fingertip. The three patches in `max/exercise2/` are independent: do them in any order, one at a time.

The tracker

PYTHON

Once, with uv installed, in the folder `max/exercise2/bridge`:

```
uv venv --managed-python --python 3.12
uv pip install -r requirements.txt
```

The environment `.venv` has its own Python; nothing else on the computer changes. Then, each time:

```
.venv/bin/python body2osc.py
```

A window shows the camera image with the points. The script sends `/hand/raw`, `/hand/present` and `/camera/aspect`; with `--pose`, `/pose/raw` (99 numbers).

github.com/torinmb/mediapipe-touchdesigner.

TOUCHDESIGNER

Torin Blankensmith's MediaPipe plugin runs the same models inside TouchDesigner, on the graphics card, with nothing else to install. Its hand-tracking component gives the 21 points; sent with an OSC Out operator to port 9001 as `/hand/raw`, they reach the same Max patches.

Useful if you already work in TouchDesigner, and the route we take in the summer semester.

EXERCISE 2

Part A: poses trigger sounds

1. Open `ex2a-hand-poses.maxpat`, start the tracker, check that the 63 features move when the hand moves.
2. Choose a label (*rest*, *fist*, *open*, *point*, or your own), hold the pose and click *add example*. Record five to ten examples per label, changing slightly the angle and the distance of the hand each time.
3. Click *train*: `fluid.normalize~` scales the dataset, `fluid.knnclassifier~` stores it with the labelset.
4. Switch on *play*. Every frame is scaled (`transformpoint`) and classified (`predictpoint`); a new label is accepted after five equal frames and triggers the sound file assigned to it.
5. Make it fail: a pose between two labels, the hand at the edge of the frame, another person's hand. Correct the instrument by adding examples, not by changing the code.

Part B: a map of hand shapes

1. Open `ex2b-hand-map.maxpat`. Switch on *record* and move the hand slowly through many shapes for about 30 seconds: the patch adds a point every 100 ms, about 300 in all.
2. Click *map*: `fluid.umap~` reduces the 63 features to 2, `fluid.normalize~` scales the map to 0–1, and `fluid.plotter` shows it.
3. Switch on *play*. The current hand shape is placed into the map with `transformpoint` and drawn as a highlighted point; its x and y control the carrier frequency and the index of the FM oscillator of Exercise 1.
4. Fit the map again with `numneighbours` 5, then 50, and compare the shapes of the map and how the sound changes when you move.

Part C: following a gesture

1. Open `ex2c-gesture-follower.maxpat`. It uses the position of the index fingertip (point 8) in the image: two features, x and y.
2. Choose example 1, switch on *record*, draw a gesture in the air (a circle, a wave, a letter), switch it off. Do the same for examples 2 and 3 with different gestures.
3. Click *train*: `mubu.hhmm` fits one model per gesture.
4. Switch on *follow*. The likeliest gesture selects one of three sound files; its progress, 0 to 1, sets the playback position. Perform the gestures slowly, fast, halfway and backwards.
5. Change `states`, the number of states per gesture (10 in the patch): fewer states follow more roughly but recognise more reliably.

EXERCISE 2

Variations

Change	Observe
A: raw coordinates instead of features	the label depends on where you stand
A: `numneighbours 1`, then `10`	jumps between labels, then hesitation at the borders
A: `fluid.mlpclassifier~` on the same data	borders between labels move
B: `fluid.pca~` instead of UMAP	a smoother map that separates the shapes less
B: record the examples of part A, colour them by label	whether the labels form islands in the map
C: the wrist instead of the fingertip; more states	which movements the model can tell apart
A + Exercise 1: features as input of `fluid.mlpregressor~`	a continuous instrument played with hand shapes

EXERCISE 2

Pd and Plugdata

The tracker sends ordinary OSC: in Pd, `netreceive -u -b 9001` → `oscparse` → `list trim` → `route hand` → `route raw`.

Parts A and B use FluCoMa objects that exist with the same names and messages in Pd vanilla.

MuBu does not exist for Pd. For part C, send the two fingertip values to Wekinator (`/wek/inputs`, port 6448) and use its dynamic time warping: it recognises the gesture when it is complete, and does not give a progress value.

DISCUSSION

Data, model, training in Exercise 2

Data

your poses and gestures, and the 30 000 hands photographed and annotated for the tracker

Model

a convolutional tracker you cannot change, and a classifier, a map or a follower you chose

Training

which examples you recorded, which you deleted, when you stopped

In Exercise 1 every layer of the instrument was yours. Now the first model was trained by Google on images you have not seen, and decides what counts as a hand. Your instrument begins where its output begins.

Audry describes how artists intervene in training itself, for instance by changing the function that judges what a good result is. In parts A and C the errors of the classifier and of the follower are material in this sense: decide which to correct and which to keep.

Audry 2021, chapters 1–4, on training.

DISCUSSION

Whose bodies

Buolamwini and Gebru (2018) tested three commercial systems that classify the gender of faces, on a dataset balanced by skin type. For lighter-skinned men the error was at most 0.8%; for darker-skinned women it reached 34.7%. The systems had been trained and evaluated mostly on lighter-skinned faces.

The same question applies to every tracker used today: which hands, bodies and lighting conditions were in its training data, and how was its accuracy measured? For MediaPipe and YOLO the model documentation answers in part; for many tools it does not.

For a piece, test the tracker with the bodies and the room of the performance, not only with your own hand at your desk.

Buolamwini & Gebru, “Gender shades”, *Proceedings of Machine Learning Research* 81 (FAT* 2018).

Cameras, bodies and the law

GDPR, Art. 4(14), 9	Biometric data are personal data from the physical or behavioural characteristics of a person that allow or confirm their unique identification. Processing them to identify a person requires, among other conditions, explicit consent.
AI Act, Art. 5, in force since 2 Feb. 2025	Prohibited, among others: inferring emotions of people at the workplace and in educational institutions, except for medical or safety reasons; categorising people by biometric data to infer, for instance, their ethnicity, political opinions or sexual orientation.

A patch that turns the shape of a hand into sound does not identify anyone, and the tracker sends only points, not images. An installation that stores camera images of visitors, or estimates their mood from their faces, is in a different situation.

The rule on emotion recognition applies to schools and universities: a system in this building that inferred the emotions of students from their faces would be prohibited, whatever its artistic purpose.

Regulation (EU) 2016/679, Art. 4(14) and 9; Regulation (EU) 2024/1689, Art. 5(1)(f)–(g).

Questions

1. In part A, which errors of the classifier would you keep in a piece, and why?
2. The tracker decides what a hand is before your model sees anything. Is it part of your instrument, like the strings of a guitar, or part of its environment, like the room?
3. A gesture follower reacts while you move. Does it change how you move, and is that a constraint or a technique?

DISCUSSION

Before the next block

- Read Audry, *Art in the Age of Machine Learning*, chapters 5–7: evolutionary, shallow and deep learning.
- Finish one part of Exercise 2 and record one minute of playing it.
- Block 3 generates sequences of notes. Bring two or three MIDI files of your own music, or of music you know well; the models of block 3 will be trained on them.

References

Audry, S. (2021). *Art in the Age of Machine Learning*. MIT Press.
Bazarevsky, V., et al. (2020). BlazePose: on-device real-time body pose tracking. arXiv:2006.10204.
Bevilacqua, F., et al. (2010). Continuous realtime gesture following and recognition. In *Gesture in Embodied Communication and Human-Computer Interaction*, LNCS 5934. Springer.
Buolamwini, J., & Gebru, T. (2018). Gender shades. *Proceedings of Machine Learning Research*, 81.
Casiez, G., Roussel, N., & Vogel, D. (2012). 1 € filter: a simple speed-based low-pass filter for noisy input in interactive systems. *CHI*.
Fix, E., & Hodges, J. L. (1951). *Discriminatory analysis: nonparametric discrimination*. USAF School of Aviation Medicine.
Françoise, J., Schnell, N., Borghesi, R., & Bevilacqua, F. (2014). Probabilistic models for designing motion and sound relationships. *NIME*.
Fukushima, K. (1980). Neocognitron. *Biological Cybernetics*, 36.
Hubel, D. H., & Wiesel, T. N. (1962). Receptive fields, binocular interaction and functional architecture in the cat's visual cortex. *Journal of Physiology*, 160.
LeCun, Y., et al. (1989). Backpropagation applied to handwritten zip code recognition. *Neural Computation*, 1(4).
Lin, T.-Y., et al. (2014). Microsoft COCO: common objects in context. *ECCV*.
McInnes, L., Healy, J., & Melville, J. (2018). UMAP: uniform manifold approximation and projection for dimension reduction. arXiv:1802.03426.
Rabiner, L. R. (1989). A tutorial on hidden Markov models and selected applications in speech recognition. *Proceedings of the IEEE*, 77(2).
Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: unified, real-time object detection. *CVPR*.
Sakoe, H., & Chiba, S. (1978). Dynamic programming algorithm optimization for spoken word recognition. *IEEE Transactions on ASSP*, 26(1).
Santello, M., Flanders, M., & Soechting, J. F. (1998). Postural hand synergies for tool use. *Journal of Neuroscience*, 18(23).
Schnell, N., et al. (2009). MuBu & friends: assembling tools for content based real-time interactive audio processing in Max/MSP. *ICMC*.
Shotton, J., et al. (2011). Real-time human pose recognition in parts from single depth images. *CVPR*.
Zeiler, M. D., & Fergus, R. (2014). Visualizing and understanding convolutional networks. *ECCV*.
Zhang, F., et al. (2020). MediaPipe Hands: on-device real-time hand tracking. arXiv:2006.10214.

MediaPipe **Ultralytics YOLO** **MuBu** **FluCoMa** **MediaPipe for TouchDesigner**